

BinX TECH • AI & MACHINE LEARNING INTERNSHIP PROGRAM

WEEK 4

EVALUATION, TUNING & PIPELINES

Cross-Validation, Bias-Variance, Feature Engineering, GridSearchCV & Scikit-learn Pipelines

The second week of Phase 2: turning a model that runs into a model that is trustworthy. Interns learn to evaluate models reliably with cross-validation, diagnose overfitting and underfitting, engineer better features, tune hyperparameters with GridSearchCV, and package the whole workflow into a clean, leakage-free Scikit-learn pipeline.

PHASE 2 • 40 HOURS • 5 TRAINING DAYS • HANDS-ON

Prepared by BinX Tech
Palestine | Nablus

BinX

TECH

1. Week 4 Overview

Week	Week 4 of 10 — Phase 2: Core ML Training
Total Hours	40 hours (full-time track) / 20 hours (part-time track, Weeks 7-8 combined)
Format	On-site / Remote / Hybrid — all work in Jupyter notebooks committed to GitHub
Focus	Model evaluation: train/val/test splits, cross-validation, bias-variance, feature engineering, GridSearchCV, and Scikit-learn pipelines
Prerequisite	Weeks 1-3 completed: Python/EDA fluency, math foundations, supervised learning with Scikit-learn
Mentor Supervision	Daily check-in; the Day 5 tuned-pipeline mini-project is the second graded model deliverable

2. Week 4 Learning Objectives

- Explain the purpose of the train / validation / test split and why a validation set is needed beyond a test set.
- Evaluate a model reliably with k-fold cross-validation instead of a single split.
- Diagnose overfitting and underfitting using the bias-variance trade-off.
- Engineer new features and correctly encode and scale existing ones to improve model performance.
- Tune hyperparameters systematically with GridSearchCV.
- Package preprocessing and modeling into a single leakage-free Scikit-learn Pipeline.

3. Daily Schedule

Day	Hours	Topic Focus
Day 1	8 hrs	Train/validation/test splits; why a single split is not enough
Day 2	8 hrs	Cross-validation: k-fold, stratified k-fold, cross_val_score
Day 3	8 hrs	Bias-variance trade-off; diagnosing overfitting and underfitting
Day 4	8 hrs	Feature engineering; hyperparameter tuning with GridSearchCV
Day 5	8 hrs	Scikit-learn Pipelines; tuned end-to-end pipeline mini-project

4. Day-by-Day Curriculum

Day 1 — Train / Validation / Test Splits — 8 hours

Learning Objectives

- Explain why a validation set is needed in addition to a test set.
- Create a correct three-way split in Scikit-learn.
- Explain why tuning against the test set produces misleading results.

Key Topics

- The problem with tuning against a single test set
- The three-way split: train, validation, test
- Creating a three-way split in code
- Why one validation set can still mislead (motivating cross-validation)

Lesson Content

1.1 The Problem With a Single Test Set

In Week 3 the data was split into train and test. But there is a hidden trap: as soon as you start tuning a model by repeatedly checking the test set, you begin fitting your decisions to that specific test set. The test score stops being an honest estimate of real-world performance, because information about it has leaked into your choices.

1.2 The Three-Way Split

The professional solution is three sets, each with one job:

Set	Purpose	When It's Used
Training set	The model learns its parameters from this	During .fit()
Validation set	Tune choices (model, hyperparameters, features) against this	During development and tuning
Test set	Final, one-time, honest performance estimate	Once, at the very end — never touched during tuning

The rule is strict: the test set is opened exactly once, after every decision is final. If you tune against the test set, you no longer have an honest estimate of how the model performs on truly new data.

1.3 Creating the Split in Code

A three-way split is done with two calls to `train_test_split`: first carve off the test set, then split the remaining data into train and validation.

```
from sklearn.model_selection import train_test_split
# 1) hold out 20% as the final test set
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
# 2) split the rest into train (75%) and validation (25%)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=42)
```

Typical proportions: 60% train / 20% validation / 20% test. The exact numbers matter less than the discipline of keeping

the test set untouched until the end.

1.4 Why This Isn't Always Enough

A single validation set has its own weakness: if it happens to be an unusual slice of the data, your tuning decisions will be based on luck rather than signal. On smaller datasets especially, one validation split can be misleading — which is exactly the problem cross-validation (Day 2) solves.

Hands-On Lab: Building a Three-Way Split

- Step 1: Take a Week 3 dataset and create a 60/20/20 train/validation/test split with a fixed `random_state`.
- Step 2: Train a model on the training set and tune one setting by checking the validation set only.
- Step 3: Evaluate the final model on the test set exactly once and report the score.
- Step 4: In Markdown, explain what would go wrong if you had tuned against the test set instead.

Tools Used

Scikit-learn (`train_test_split`) • Pandas • Jupyter Notebook

Day 2 — Cross-Validation — 8 hours

Learning Objectives

- Explain how k-fold cross-validation produces a reliable performance estimate.
- Run cross-validation with `cross_val_score` and interpret the mean and standard deviation.
- Explain why stratified k-fold matters for classification.

Key Topics

- Why cross-validation beats a single validation split
- How k-fold works: rotating folds
- `cross_val_score`: mean and standard deviation
- Stratified k-fold for balanced classification folds

Lesson Content

2.1 What Cross-Validation Does

k-fold cross-validation replaces one lucky-or-unlucky validation split with k of them. It splits the training data into k equal parts ("folds"), then trains k times — each time using a different fold as the validation set and the other k-1 folds for training. Averaging the k scores gives a far more stable, trustworthy estimate of performance than any single split.

2.2 How k-Fold Works (k = 5)

Round	Trains On	Validates On
1	Folds 2,3,4,5	Fold 1
2	Folds 1,3,4,5	Fold 2
3	Folds 1,2,4,5	Fold 3
4	Folds 1,2,3,5	Fold 4
5	Folds 1,2,3,4	Fold 5

Every data point is used for validation exactly once and for training $k-1$ times, so no data is wasted and no single split can dominate the result. $k=5$ or $k=10$ are the common choices.

2.3 cross_val_score

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X_train, y_train, cv=5, scoring="f1")
print(scores)           # one score per fold
print(scores.mean())    # the reliable average
print(scores.std())     # how much the score varies across folds
```

The mean is your performance estimate; the standard deviation tells you how stable it is. A high mean with a low standard deviation is a model you can trust; a high mean with a high standard deviation may just be lucky on some folds.

2.4 Stratified k-Fold for Classification

On classification problems — especially imbalanced ones — plain k-fold can accidentally create folds with very different class proportions. Stratified k-fold fixes this by preserving the original class balance in every fold, and Scikit-learn applies it automatically when you cross-validate a classifier. This directly addresses the imbalance problem raised in Week 3.

Hands-On Lab: Cross-Validating a Model

- Step 1: Take a Week 3 model and evaluate it with 5-fold cross-validation using `cross_val_score`.
- Step 2: Report the mean and standard deviation of the scores across folds.
- Step 3: Compare the cross-validated estimate to the single-split score from Day 1 and explain any difference.
- Step 4: For a classification task, confirm stratified folds are used and explain why that matters here.

Tools Used

Scikit-learn (`cross_val_score`, `StratifiedKFold`) • Pandas • Jupyter Notebook

Day 3 — Bias-Variance & Diagnosing Model Fit — 8 hours

Learning Objectives

- Distinguish underfitting from overfitting by their symptoms.
- Explain the bias-variance trade-off and its role in tuning.
- Diagnose model fit from the train-vs-validation score gap and apply regularization.

Key Topics

- Underfitting (high bias) vs. overfitting (high variance)
- The bias-variance trade-off
- Diagnosing fit from the train-vs-validation gap
- Regularization: Ridge (L2) and Lasso (L1)

Lesson Content

3.1 The Two Ways a Model Fails

Every model failure is one of two kinds, and the fix for each is the opposite of the fix for the other — so diagnosing which one you have is the essential first step.

	Underfitting (High Bias)	Overfitting (High Variance)
--	--------------------------	-----------------------------

Symptom	Poor on both training AND validation data	Great on training, poor on validation data
Cause	Model too simple to capture the pattern	Model too complex; memorized the training data
Fix	Add features, use a more powerful model	Simplify the model, add data, regularize

3.2 The Bias-Variance Trade-off

Bias is error from wrong assumptions (too simple); variance is error from sensitivity to the specific training data (too complex). As you make a model more complex, bias falls but variance rises. The goal is the sweet spot in the middle — complex enough to capture the real pattern, simple enough to generalize. This trade-off is the single most important concept in practical model tuning.

3.3 Diagnosing With the Train-vs-Validation Gap

The gap between training and validation scores is the diagnostic tool:

Training Score	Validation Score	Diagnosis
Low	Low	Underfitting — model too simple
High	Much lower	Overfitting — model memorized training data
High	High (small gap)	Good fit — this is the target

3.4 Regularization

Regularization is the standard cure for overfitting: it adds a penalty for model complexity, discouraging the model from relying too heavily on any single feature. In linear models this appears as Ridge (L2) and Lasso (L1) regression; Lasso can even shrink weak features' weights to exactly zero, effectively selecting features.

```
from sklearn.linear_model import Ridge, Lasso
ridge = Ridge(alpha=1.0)    # L2: shrinks all weights toward zero
lasso = Lasso(alpha=0.1)    # L1: can zero out weak features entirely
```

The alpha parameter controls the penalty strength — larger alpha means simpler model. Choosing alpha well is itself a hyperparameter-tuning problem, which motivates GridSearchCV on Day 4.

Hands-On Lab: Diagnosing and Fixing Model Fit

- Step 1: Deliberately overfit a model (e.g. a very deep decision tree) and confirm the large train-vs-validation gap.
- Step 2: Deliberately underfit a model (e.g. an overly simple one) and confirm both scores are low.
- Step 3: Apply regularization (or reduce complexity) to the overfit model and show the gap shrink.
- Step 4: Document each diagnosis and fix with the score evidence in Markdown.

Tools Used

Scikit-learn (Ridge, Lasso, DecisionTree) • Pandas • Matplotlib • Jupyter Notebook

Day 4 — Feature Engineering & Hyperparameter Tuning — 8 hours

Learning Objectives

- Engineer new features and apply appropriate transformations.
- Distinguish hyperparameters from learned parameters.
- Tune a model systematically with GridSearchCV and cross-validation.

Key Topics

- Why feature engineering often beats model choice
- Common techniques: feature creation, binning, encoding, datetime, scaling
- Hyperparameters vs. parameters
- GridSearchCV: grid, cross-validation, best_params_
- RandomizedSearchCV for large grids

Lesson Content

4.1 Why Feature Engineering Often Beats Model Choice

A common professional truth: better features usually improve results more than a fancier model. Feature engineering is the process of creating, transforming, and selecting the input variables the model learns from — and it is where domain understanding turns into predictive power.

4.2 Common Feature Engineering Techniques

Technique	What It Does	Example
Creating features	Combine existing columns into a more informative one	price_per_sqm = price / area
Binning	Group a continuous variable into ranges	age → child / adult / senior
One-hot encoding	Convert categories into numeric columns	city → city_Nablu, city_Ramallah, ...
Datetime extraction	Pull signal from dates	order_date → day_of_week, month
Scaling	Put numeric features on a comparable range	StandardScaler, MinMaxScaler

4.3 Hyperparameters vs. Parameters

A parameter is learned by the model during training (e.g. a regression coefficient). A hyperparameter is set by you before training (e.g. a tree's max_depth, k in k-NN, alpha in Ridge). Finding good hyperparameters is a search problem, and doing it by hand is slow and unsystematic.

4.4 GridSearchCV

GridSearchCV automates hyperparameter tuning: you define a grid of values to try, and it trains and cross-validates a model for every combination, then reports the best one. It combines the search with cross-validation, so the chosen hyperparameters are selected on a reliable estimate rather than a single lucky split.

```
from sklearn.model_selection import GridSearchCV
param_grid = {
    "n_estimators": [100, 200],
    "max_depth": [5, 10, None],
}
grid = GridSearchCV(RandomForestClassifier(random_state=42),
                    param_grid, cv=5, scoring="f1")
grid.fit(X_train, y_train)
print(grid.best_params_) # the winning combination
print(grid.best_score_) # its cross-validated score
best_model = grid.best_estimator_
```

Grid search cost grows fast: 2 values × 3 values × 5 folds = 30 model fits. For large grids, RandomizedSearchCV samples a subset of combinations and is far faster.

Hands-On Lab: Engineering Features & Tuning

- Step 1: Create at least two new engineered features for a provided dataset and justify each in Markdown.
- Step 2: Define a hyperparameter grid for a model from Week 3 (e.g. random forest).
- Step 3: Run GridSearchCV with 5-fold cross-validation and report the best parameters and score.
- Step 4: Compare the tuned model's cross-validated score against the untuned baseline from Week 3.
- Step 5: Document which engineered feature and which hyperparameter mattered most.

Tools Used

Scikit-learn (GridSearchCV) • Pandas • Jupyter Notebook

Day 5 — Scikit-learn Pipelines & Tuned Mini-Project — 8 hours

Learning Objectives

- Build a Pipeline that chains preprocessing and modeling leak-free.
- Use ColumnTransformer to preprocess numeric and categorical columns differently.
- Tune an entire pipeline with GridSearchCV and evaluate it correctly.

Key Topics

- Why pipelines prevent data leakage
- Building a Pipeline: preprocessing + model in one object
- ColumnTransformer for mixed numeric/categorical data
- Tuning a whole pipeline with GridSearchCV
- Assembling a professional, leak-free workflow

Lesson Content

5.1 Why Pipelines Exist

So far, preprocessing (scaling, encoding) and modeling have been separate manual steps — which is exactly how data leakage sneaks in: if you scale the whole dataset before splitting, or fit a scaler on validation folds during cross-validation, test information leaks into training. A Pipeline chains preprocessing and modeling into one object that applies every step in the correct order, automatically and leak-free.

5.2 Building a Pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", RandomForestClassifier(random_state=42)),
])
pipe.fit(X_train, y_train)      # scales, then trains – in one call
pipe.predict(X_test)           # scales test data the same way, then predicts
```

The key benefit: when you call `pipe.fit()`, the scaler is fit on the training data only; when you cross-validate the pipeline, each fold is scaled using only that fold's training portion. Leakage becomes structurally impossible.

5.3 ColumnTransformer for Mixed Data

Real datasets have both numeric and categorical columns needing different treatment. ColumnTransformer applies different preprocessing to different column groups — for example, scaling numeric columns while one-hot encoding categorical ones — and slots directly into a pipeline.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
```

```
pre = ColumnTransformer([
    ("num", StandardScaler(), numeric_cols),
    ("cat", OneHotEncoder(), categorical_cols),
])
pipe = Pipeline([("pre", pre), ("model", RandomForestClassifier())])
```

5.4 Tuning a Whole Pipeline

GridSearchCV can tune an entire pipeline at once — searching over both preprocessing options and model hyperparameters together, using a double-underscore to name a step's parameter. This is how professional ML code is structured: one object, cross-validated and tuned end to end, with no leakage anywhere.

```
param_grid = {"model__n_estimators": [100, 200],
              "model__max_depth": [5, 10]}
grid = GridSearchCV(pipe, param_grid, cv=5, scoring="f1")
grid.fit(X_train, y_train)
```

5.5 The Week 4 Mini-Project

Week 4 closes with a single tuned pipeline: EDA-informed feature engineering, a ColumnTransformer for mixed data, a model, all inside one Pipeline, tuned with GridSearchCV, cross-validated, and evaluated once on a held-out test set. This is the exact structure the Phase 3 capstone project requires — professional, reproducible, and leak-free.

Hands-On Lab: Tuned End-to-End Pipeline

- Step 1: Build a Pipeline with a ColumnTransformer handling numeric (scaling) and categorical (encoding) columns.
- Step 2: Add the engineered features from Day 4 into the workflow.
- Step 3: Tune the full pipeline with GridSearchCV and 5-fold cross-validation.
- Step 4: Evaluate the final tuned pipeline once on the held-out test set and report the metric against a baseline.
- Step 5: Commit the finished pipeline notebook to GitHub with a clear commit message.

Tools Used

Scikit-learn (Pipeline, ColumnTransformer, GridSearchCV) • Pandas • Jupyter Notebook • Git & GitHub

5. Week 4 Deliverables

By the end of Week 4, every intern must submit the following to their mentor and GitHub repository:

- A three-way split notebook demonstrating correct train/validation/test discipline.
- A cross-validation notebook reporting mean and standard deviation across folds, with stratification for classification.
- A bias-variance notebook showing a diagnosed overfit and underfit case, each with its fix and score evidence.
- A feature-engineering and GridSearchCV notebook with engineered features and reported best hyperparameters.
- The Week 4 tuned pipeline mini-project: a leak-free Pipeline + ColumnTransformer, tuned and evaluated on held-out data.
- All Week 4 notebooks committed to the intern's GitHub repository with clear commit messages.

6. Week 4 Evaluation Criteria

Scored by the assigned mentor at the end of Week 4 using the criteria below (extracted from the program's 100-point internal rubric).

Criterion	50-69 Developing	70-84 Proficient	85-100 Excellent
Evaluation rigor	Single split, no cross-validation	Uses cross-validation and a held-out test set correctly	Reports mean $\pm$ std, stratifies, keeps test set untouched
Bias-variance diagnosis	Cannot reliably diagnose fit	Correctly diagnoses overfit/underfit and applies a fix	Precise diagnosis with regularization/complexity reasoning
Feature engineering	Little or no feature work	Creates and justifies useful new features	Insightful, domain-informed features that improve results
Tuning & pipelines	Manual tuning, leaky preprocessing	GridSearchCV + a working leak-free pipeline	Whole pipeline tuned end-to-end, fully leakage-free
Notebook quality & Git workflow	Sporadic commits, weak narrative	Regular commits, clear Markdown narrative	Consistent, descriptive, reproducible, well-organized
Attendance & punctuality	3-6 absences	1-2 absences, on time	Perfect attendance, proactive

7. Technical Stack — Week 4

Evaluation	train_test_split, cross_val_score, StratifiedKFold
Tuning	GridSearchCV, RandomizedSearchCV
Regularization	Ridge (L2), Lasso (L1)
Pipelines	Pipeline, ColumnTransformer, StandardScaler, OneHotEncoder
Environment	Python 3.10+, Pandas, Scikit-learn, Jupyter Notebook, Git & GitHub

8. Notes & Best Practices

This week separates interns who can run a model from those who can trust one. The recurring theme is honest evaluation: cross-validate instead of trusting one split, diagnose fit before tuning, and use pipelines so leakage is structurally impossible. The Day 5 tuned pipeline is the exact workflow required for the Phase 3 capstone — master it here.